Human-Supervised CFAR Autotuning for Maritime Sensing

N. Kirk, J.C. Vaught, D. Cahl, Y. Wang

University of South Carolina
Columbia, SC 29208

AIAA Region II Student Conference 2026

Motivation: CFAR Tuning Challenges

Problem: CFAR detectors require scene-specific parameter tuning

Real-world challenges:

- Heavy-tailed clutter (K-distribution, Pareto models)
- Terrain transitions and multipath interference
- Spatially nonhomogeneous background statistics
- No universal parameter set works across environments

Current practice: Manual trial-and-error

- Slow, subjective, not reproducible
- Operators adjust until display "looks right"
- No quantitative optimization objective

Our approach: Formalize operator knowledge as ground truth and optimize systematically

Background: CFAR Detection Variants

All variants use 2D sliding window with guard cells N_g and reference cells N_r

Threshold multiplier (exponential clutter):

$$\alpha = N\left(P_{\mathrm{FA}}^{-1/N} - 1\right)$$

CA-CFAR

- Mean of all reference cells
- Optimal for homogeneous clutter

$$Z^{\mathrm{CA}} = \frac{1}{N}\left(S_{\mathrm{outer}} - S_{\mathrm{inner}}\right)$$

GO-CFAR

- Greatest of two half-windows
- Suppresses false alarms at edges

$$Z^{\mathrm{GO}} = \max(\bar{Z}_1, \bar{Z}_2)$$

SO-CFAR

- Smallest of two half-windows
- Maintains sensitivity near edges

$$Z^{\mathrm{SO}} = \min(\bar{Z}_1, \bar{Z}_2)$$

Detection rule:

$$D_t(i,j) = \mathbb{1}[I_t(i,j) > \alpha \cdot Z_{i,j}]$$

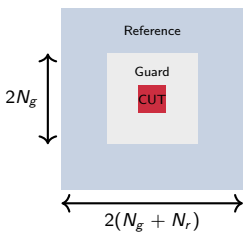
Problem Formulation

Goal: Find optimal CFAR variant $v \in \{\text{CA, GO, SO}\}$ and parameters $\boldsymbol{\theta} = (N_g, N_r, P_{\text{FA}})$

Key insight: Integral images

- Pre-compute summed area table
- $O(1)$ query for any rectangle
- Enables fast noise map computation

2D Sliding Window:



Optimization challenge:

- Parameter space: $N_g \in [1, 10]$, $N_r \in [5, 30]$, $P_{\text{FA}} \in [10^{-6}, 10^{-2}]$
- Noise map depends on (N_g, N_r) only
- Loop hoisting: compute once per (N_g, N_r), reuse across all P_{FA}

Speedup factor: Number of P_{FA} values tested

Annotation and Scoring Function

Blob tracing annotation:

- Operator clicks on detected target
- 8-connected flood fill captures exact shape
- Ground truth blob $\mathcal{B}_k$ = pixel set

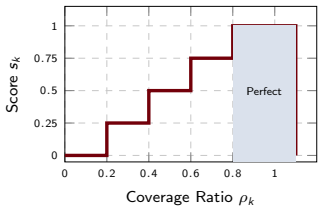
Coverage ratio:

$$\rho_k = \frac{\sum_{p \in \mathcal{B}_k} D_t(p)}{|\mathcal{B}_k|}$$

Score mapping:

$$s_k = \begin{cases} 1.00 & \text{if } 0.8 \leq \rho_k \leq 1.1 \\ 0.75 & \text{if } 0.6 \leq \rho_k < 0.8 \\ 0.50 & \text{if } 0.4 \leq \rho_k < 0.6 \\ 0.25 & \text{if } 0.2 \leq \rho_k < 0.4 \\ 0.00 & \text{if } \rho_k < 0.2 \end{cases}$$

Scoring function visualization:



Frame score (weighted):

$$S_{\text{frame}} = 0.7 \cdot \min_k s_k + 0.3 \cdot \frac{1}{K} \sum_{k=1}^{K} s_k$$

Heavy penalty for missing any target

Grid Search with Loop Hoisting

Algorithm 1 Optimized Grid Search

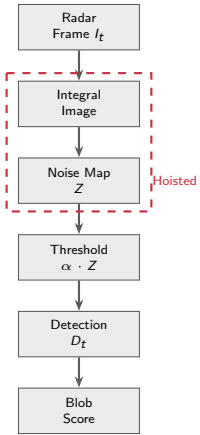
Input: frames, blobs, variant
Pre-compute integral images
for each (N_g, N_r) **do**
    Compute noise map $Z \leftarrow$ expensive
    for each P_{FA} **do**
        $\alpha \leftarrow N(P_{\mathrm{FA}}^{-1/N} - 1)$
        $D_t \leftarrow \mathbb{1}[I_t > \alpha Z]$
        Score detections vs. blobs
    end for
end for
Return sorted results

Temporal stability:

$$\text{Stability} = \frac{1}{K} \sum_{k=1}^{K} \frac{c_k}{T}$$

where c_k = frames detecting target k

Processing pipeline:



Performance: ∼500 configs in <10 seconds

System Architecture: CFAR Studio

Fully client-side web application – no installation, no server, no data upload

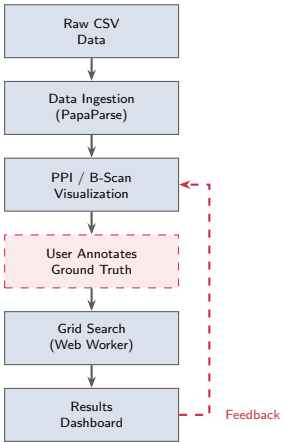
Technology stack:

- React 19 + TypeScript
- Vite build system
- Plotly.js for interactive visualization
- PapaParse for CSV ingestion
- Web Worker for grid search

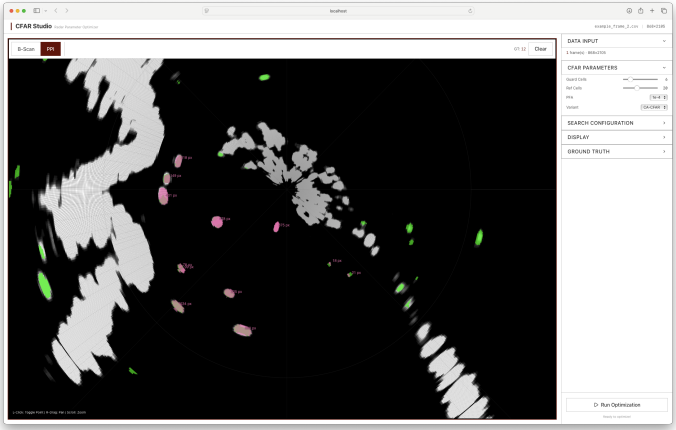
Key features:

- PPI and B-scan visualization
- Multiple colormaps
- Interactive blob annotation
- Multi-frame temporal analysis
- Real-time progress updates

Processing pipeline:

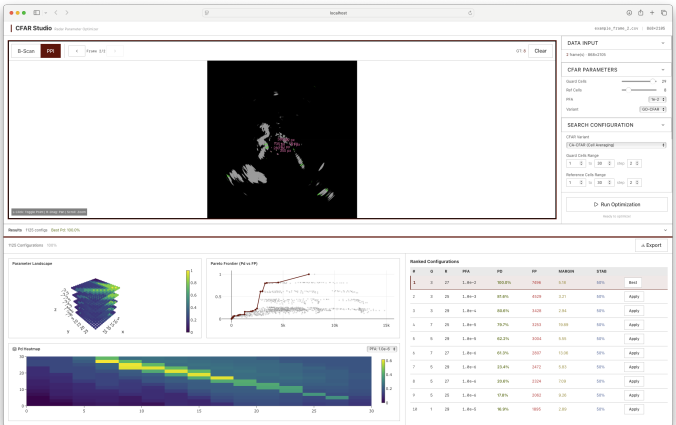


Results: PPI View with Detections



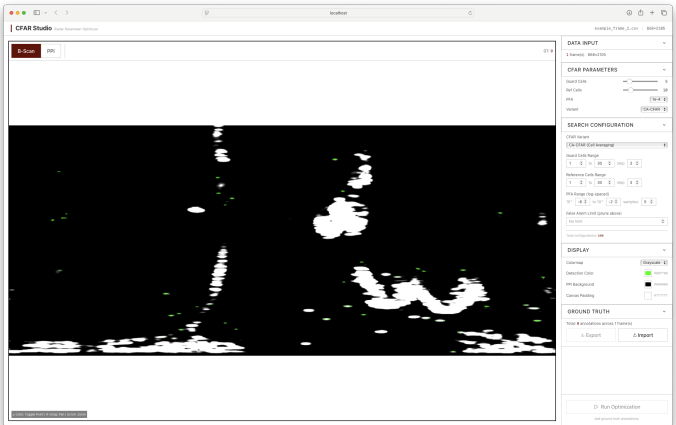
Data: Furuno DRS4D NXT X-band (9.41 GHz) radar at Lake Murray, SC
Platform: Raspberry Pi 5 digitization and recording
Environment: Mixed water, shoreline structures, fixed infrastructure

Results: Full CFAR Studio Interface



Interactive parameter controls (left), PPI display (center), optimization results dashboard (right)

Results: B-Scan View



Range vs. azimuth representation reveals clutter structure and detection behavior along radar spokes

Results: Optimization Findings

Parameter space: Grid search over $\sim$500 configurations in $<$10 seconds

Guard cells N_g:

- $N_g \geq 3$ improves scores for extended targets
- Prevents target self-contamination of noise estimate
- Point targets less sensitive to guard cell count

Reference cells N_r:

- Optimal range: $N_r \in [10, 20]$
- Balance: noise estimation accuracy vs. spatial homogeneity
- Large $N_r > 25$ degrades performance at terrain transitions

False alarm probability P_{FA}:

- Scene-dependent optimal value
- Open water: lower P_{FA} acceptable
- Near clutter edges: moderate P_{FA} maintains detection

Variant comparison:

- **GO-CFAR:** fewer false alarms, conservative
- **SO-CFAR:** higher detection, more false alarms
- **CA-CFAR:** best overall in homogeneous scenes

Temporal stability: Top configs achieve $>$0.9 across all frames

Discussion and Limitations

Strengths:

- Formalizes operator knowledge as quantitative objective
- Exhaustive search finds scene-specific optimal config
- Area-weighted scoring more informative than binary detection
- Temporal stability identifies consistent configs
- Zero-infrastructure deployment (browser-only)
- Open source, no licensing constraints

Applications:

- Autonomous vessel mapping and deconfliction
- Search-and-rescue in cluttered environments
- Ad-hoc deployments in unfamiliar scenes

Limitations:

- Assumes quasi-stationary targets and fixed platform
- Moving targets require frame registration or tracking
- Browser constraints: no GPU, single-threaded JS
- Practical CSV file size limit: $\sim$50 MB
- Grid search feasible for current 3-parameter space
- More variants (OS-CFAR) would benefit from Bayesian optimization

Future work:

- Extend to additional CFAR variants
- Support for moving target annotation
- GPU acceleration via WebGL/WebGPU
- Cross-sensor supervision (e.g., AIS, camera)

Questions?

Human-Supervised CFAR Autotuning
for Maritime Sensing

N. Kirk, J.C. Vaught, D. Cahl, Y. Wang

University of South Carolina